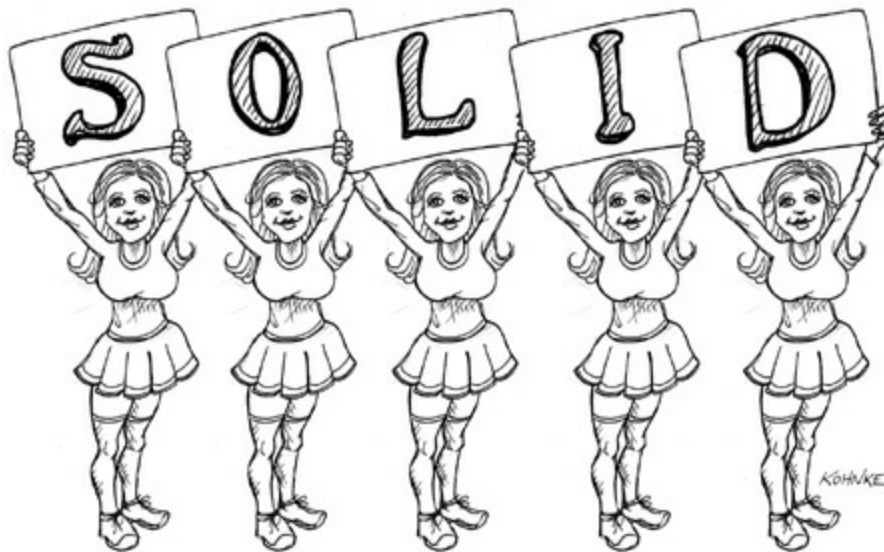

III

DESIGN PRINCIPLES



Good software systems begin with clean code. On the one hand, if the bricks aren't well made, the architecture of the building doesn't matter much. On the other hand, you can make a substantial mess with well-made bricks. This is where the SOLID principles come in.

The SOLID principles tell us how to arrange our functions and data structures into classes, and how those classes should be interconnected. The use of the word “class” does not imply that these principles are applicable only to object-oriented software. A class is simply a coupled grouping of functions and data. Every software system has such groupings, whether they are called classes or not. The SOLID principles apply to those groupings.

The goal of the principles is the creation of mid-level software structures that:

- Tolerate change,
- Are easy to understand, and
- Are the basis of components that can be used in many software systems.

The term “mid-level” refers to the fact that these principles are applied by programmers working at the module level. They are applied just above the level of the code and help to define the kinds of software structures used within modules and components.

Just as it is possible to create a substantial mess with well-made bricks, so it is also possible to create a system-wide mess with well-designed mid-level components. For this reason, once we have covered the SOLID principles, we will move on to their counterparts in the component world, and then to the principles of high-level architecture.

The history of the SOLID principles is long. I began to assemble them in the late 1980s while debating software design principles with others on USENET (an early kind of Facebook). Over the years, the principles have shifted and changed. Some were deleted. Others were merged. Still others were added. The final grouping stabilized in the early 2000s, although I presented them in a different order.

In 2004 or thereabouts, Michael Feathers sent me an email saying that if I rearranged the principles, their first words would spell the word SOLID—and thus the SOLID principles were born.

The chapters that follow describe each principle more thoroughly. Here is the executive summary:

- **SRP:** The Single Responsibility Principle

An active corollary to Conway’s law: The best structure for a software system is heavily influenced by the social structure of the organization that uses it so that each software module has one, and only one, reason to change.

- **OCP:** The Open-Closed Principle

Bertrand Meyer made this principle famous in the 1980s. The gist is that for software systems to be easy to change, they must be designed to allow the behavior of those systems to be changed by adding new code, rather than changing existing code.

- **LSP:** The Liskov Substitution Principle

Barbara Liskov’s famous definition of subtypes, from 1988. In short, this principle

says that to build software systems from interchangeable parts, those parts must adhere to a contract that allows those parts to be substituted one for another.

- **ISP:** The Interface Segregation Principle

This principle advises software designers to avoid depending on things that they don't use.

- **DIP:** The Dependency Inversion Principle

The code that implements high-level policy should not depend on the code that implements low-level details. Rather, details should depend on policies.

These principles have been described in detail in many different publications¹ over the years. The chapters that follow will focus on the architectural implications of these principles instead of repeating those detailed discussions. If you are not already familiar with these principles, what follows is insufficient to understand them in detail and you would be well advised to study them in the footnoted documents.

¹. For example, *Agile Software Development, Principles, Patterns, and Practices*, Robert C. Martin, Prentice Hall, 2002, <http://www.butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod>, and [https://en.wikipedia.org/wiki/SOLID_\(object-oriented_design\)](https://en.wikipedia.org/wiki/SOLID_(object-oriented_design)) (or just google SOLID).